MIDISTRUCT V2.0 — Technical Documentation

> ***Acrosonus Mastering Studio***
> *Engine: ReaScript Lua — REAPER 6.x / 7.x*
> *Renderer: ReaImGui (ImGui_GetVersion API)*
> *License: GNU GPL v3*
> *Internal PPQ resolution:* ***240***

Table of Contents

1. Architecture Overview

MIDISTRUCT is a single-file ReaScript with no external Lua dependencies beyond the REAPER API and ReaImGui. Execution is fully event-driven through a `reaper.defer()` render loop.

```
┌──────────────────────────────────────────────────────────┐
│                    render_imgui()  [defer loop]          │
│                                                          │
│   User input ──► run_composition_engine()                │
│                        │                                 │
│                  Engine:new()                            │
│                        │                                 │
│              Engine:generate_song(chords_text)           │
│                   ├── parse_chords()                     │
│                   ├── _infer_key()                       │
│                   ├── _build_core_motif()                │
│                   ├── _apply_secondary_dominants()       │
│                   └── [bar loop]                         │
│                        ├── _gen_melody()                 │
│                        ├── _gen_counter_melody()         │
│                        ├── _gen_drums()                  │
│                        ├── _gen_bass()                   │
│                        └── _gen_pad()                    │
│                                                          │
│              write_midi_multi_track(song_data, engine)   │
│                   ├── delete_old_midistruct_tracks()     │
│                   ├── create_track() × 5                 │
│                   ├── insert_section_regions()           │
│                   ├── CreateNewMIDIItemInProj() per section │
│                   └── MIDI_InsertNote / MIDI_InsertCC    │
└──────────────────────────────────────────────────────────┘
```

Data flow summary:
`chords_text (string)` → `bars[] (chord structs)` → `song_data[] (bar structs)` → `REAPER MIDI items`

2. Module Map

Symbol	Type	Role
`RNG`	table/class	Deterministic LCG pseudo-random number generator
`Humanizer`	table/class	Timing and velocity jitter
`GROOVE_MAPS`	table	Per-style, per-step timing offset tables
`SCALES`	table	Scale interval sets (semitone arrays)
`CHORD_INTERVALS`	table	Chord quality → interval arrays
`STYLE_PRO`	table	Array of 7 genre profile records
`CHORD_PRESETS`	table	17 preset progressions
`SECTION_*`	tables	Per-section dynamics, contour, colour, push/pull constants
`Engine`	table/class	Core composition engine
`write_midi_multi_track`	function	REAPER MIDI item creation and note writing
`render_imgui`	function	ReaImGui deferred render loop
`save_settings` / `load_settings`	functions	ExtState persistence

---

3. Deterministic RNG (`RNG`)

Algorithm

Linear Congruential Generator (Lehmer / Park-Miller variant):

```
seed_n+1 = (seed_n × 1103515245 + 12345) mod 2^31
  rand()   = seed_n+1 / 2^31        → [0, 1)
```

API

```
RNG:new(seed)           -- constructor; seed defaults to os.time()
  RNG:rand()              -- float in [0, 1)
  RNG:randint(a, b)       -- integer in [a, b] inclusive
  RNG:choices(seq, weights) -- weighted random selection from seq
  RNG:gauss(mu, sigma)    -- Box-Muller Gaussian sample
  RNG:shuffle(t)          -- Fisher-Yates in-place shuffle, returns t
```

`RNG:gauss` implementation

Box-Muller transform:

```
u1 = max(1e-9, rand())       -- avoids log(0)
  u2 = rand()
  z  = sqrt(-2 × ln(u1)) × cos(2π × u2)
  return z × sigma + mu
```

Only one of the two Box-Muller outputs is used per call (no caching of the second value).

Seeding strategy

```
actual_seed = (seed == 0)
      and (floor(reaper.time_precise() × 1000) % 2147483647)
      or  seed
  if actual_seed == 0 then actual_seed = 777 end
```

The engine seed is stored in `engine.seed` and logged to history.

---

4. Humanizer

```
Humanizer:new(rng)
```

Wraps an `RNG` instance. Three methods:

`Humanizer:timing(base_tick, tightness, step, deterministic)`

Returns a humanized tick position.

Non-deterministic mode (`deterministic = false`):

```
sigma = 5.5 / clamp(tightness, 0.1, 5.0)
  return floor(base_tick + gauss(0, sigma))
```

Deterministic mode:

```
push_pull:
    step % 4 == 2   →  +18 / tightness   (off-beat 16th)
    step % 2 == 1   →  +8  / tightness   (odd 16th)
    else            →  0

  sigma_det = sigma × 0.25               (25% of free-mode sigma)
  return floor(base_tick + push_pull + gauss(0, sigma_det))
```

`tightness` values by style range from 0.6 (Lo-Fi) to 2.0 (Techno).

`Humanizer:velocity(base, spread)`

```
return clamp(floor(gauss(base, 13 × spread)), 1, 127)
```

`Humanizer:rubato(step, intensity)`

Deterministic micro-tempo perturbation based on metric position:

Condition	Offset (ticks)	Sigma
`step % 16 == 15`	−6 × intensity	2.0
`step % 8 == 7`	−3 × intensity	1.5
`step % 4 == 0`	+2 × intensity	1.0
other	0	—

Result is scaled by `PPQ / 120` before use in the MIDI writer.

---

5. Groove Maps

```
GROOVE_MAPS = {
      lofi, rnb, pop, techno, house, trap, rock
  }
```

Each map is a table indexed `[0]` to `[15]`, storing a signed tick offset applied to every event at that step position. The offset is added after all other timing calculations.

```
lofi[0]=8, lofi[1]=-15, lofi[2]=22, lofi[3]=0, lofi[4]=-25, ...
  pop[2]=5, pop[6]=8, pop[10]=5, pop[14]=8   -- nearly straight
  techno = {}                                  -- all zeros
```

Applied conditionally when `engine.options.organic_groove_maps == true`.

---

6. Music Theory Tables

`SCALES`

```
major          = {0,2,4,5,7,9,11}
  minor          = {0,2,3,5,7,8,10}
  dorian         = {0,2,3,5,7,9,10}
  phrygian       = {0,1,3,5,7,8,10}
  mixo           = {0,2,4,5,7,9,10}
  pentatonic_maj = {0,2,4,7,9}
  pentatonic_min = {0,3,5,7,10}
```

`CHORD_INTERVALS`

13 qualities supported. Examples:

```
["maj"]  = {0,4,7}
  ["min"]  = {0,3,7}
  ["m7"]   = {0,3,7,10}
  ["maj7"] = {0,4,7,11}
  ["7"]    = {0,4,7,10}
  ["9"]    = {0,4,7,10,14}
  ["dim7"] = {0,3,6,9}
  ["5"]    = {0,7}
```

`STYLE_SCALE` — default scale per style

```
pop="major", lofi="pentatonic_min", rnb="dorian",
  techno="phrygian", house="minor", trap="minor", rock="pentatonic_min"
```

`SECTION_*` constants

```
SECTION_DYN      -- dynamic multiplier: Chorus=1.00, Outro=0.48, etc.
  SECTION_CONTOUR  -- melodic shape: "rise","fall","peak","wave","flat"
  SECTION_TARGET_DEGREE  -- {degree, oct_offset} per section
  SECTION_PUSH_PULL      -- bar-level PPQ offset: Chorus=-12, Verse=+10, etc.
  SECTION_REGION_COLOR   -- {r,g,b} for REAPER region colour
```

`SIGNATURE_INTERVALS`

Per-style pair of signature intervals used to seed the motif contour:

```
pop={7,9,5}, lofi={3,5,7}, rnb={9,7,12},
  techno={1,6,11}, house={7,5,3}, trap={3,10,7}, rock={7,5,10}
```

---

7. Style Profiles (`STYLE_PRO`)

Each record in the `STYLE_PRO` array defines a complete genre environment:

```
{
      name             string    -- display name
      bpm              number    -- beats per minute
      swing            integer   -- swing ticks (0-100 scale, applied as fraction of PP
  Q)
      hat_swing_extra integer    -- additional hi-hat swing offset
      kick             table     -- step indices {0..15} where kick fires
      snare            table     -- step indices for snare
      energy           float     -- base velocity scale [0.0, 1.0]
      tightness        float     -- timing sigma divisor [0.6, 2.0]
      legato           float     -- note duration factor [0.65, 1.10]
      scale_type       string    -- key into SCALES / STYLE_SCALE / GROOVE_MAPS
      rubato_intensity float     -- rubato amplitude multiplier
      use_ride         boolean   -- true = ride cymbal replaces hi-hat
      hat_open_steps   table     -- steps where open hat fires instead of closed
  }
```

Style blending (`blend_styles`)

```
function blend_styles(s1, s2, ratio)
```

- All **numeric fields** interpolate linearly: `result = s1[k] × (1 − r) + s2[k] × r` where `r = ratio / 100`.
- **Table and boolean fields** (`kick` , `snare` , `scale_type` , `use_ride` , `hat_open_steps`) switch hard at `r > 0.5` .

---

8. Engine Initialization & Style Blending

```
Engine:new(s1_idx, s2_idx, mix, comp, seed, options)
```

Parameter	Type	Description
`s1_idx`	integer	Primary style index (1-based into `STYLE_PRO`)
`s2_idx`	integer or nil	Secondary style index; 0 or nil = no blend
`mix`	integer	Blend ratio 0–100
`comp`	integer	Complexity 1–10, clamped
`seed`	integer	RNG seed; 0 means clock-derived
`options`	table	Boolean flags map (all option keys)

State fields initialized on `Engine:new` :

```
engine.rng              -- RNG instance
  engine.human            -- Humanizer instance
  engine.cfg              -- blended style record
  engine.tightness        -- alias to cfg.tightness
  engine.legato           -- alias to cfg.legato
  engine._prev_voicing    -- nil (chord voice-leading state)
  engine._core_motif      -- nil (set by _build_core_motif)
  engine._global_scale    -- nil (set by _infer_key)
  engine._global_root     -- nil
  engine._scale_intervals -- nil
  engine._scale_key       -- nil
  engine._key_name        -- ""
  engine._modulation_st   -- 2 (semitones for Bridge modulation)
  engine._sig_intervals   -- SIGNATURE_INTERVALS[scale_type]
  engine._hook_play_count -- 0
  engine._hook_variation  -- 0  (cycles 0→1→2→3→1→...)
  engine._phrase_cache    -- {}
  engine._melody_play_streak -- 0
```

---

9. Chord Parser (`Engine:parse_chords`)

```
bars = engine:parse_chords(chords_text)
```

Input: space-separated chord tokens `ROOT[QUALITY][:DURATION]`

Steps:

1. Strip newlines; tokenize on whitespace.
2. For each token: extract `root_str` + `quality_str` + `dur` via pattern `"([^:]+):?(%d*)"` then `"^([A-G][#b]?)(.*)"`.
3. Map root to pitch class via `NOTE_MAP`.
4. Parse quality string (case-insensitive, longest-match priority): m7 > maj7 > m9 > add9 > 9 > 7 > sus4 > sus2 > dim7 > dim > aug > 5 > min/m > (default major).
5. Compute pitch-class set: `pcs[i] = (root_pc + interval[i]) % 12`.
6. **Modal Interchange** (if `options.modal_interchange`): with 35% probability, replace the penultimate or final subdominant chord (root = key_root + 5) with its parallel minor quality.
7. Repeat the chord record `dur` times → one entry per bar.

Output: array of `{root, pcs, quality, name}` records.
Quality detection order matters — "m7" is tested before "7" and "m" to prevent false positives.

10. Key Inference (`Engine:_infer_key`)

```
engine:_infer_key(bars)
```

Algorithm:

```
pc_weight[0..11] = 0

  for each bar:
      for each pc in bar.pcs:   pc_weight[pc] += 1
      pc_weight[bar.root]       += 2          -- root gets extra weight

  best_root = argmax(pc_weight)
```

Scale selection:

```
style_default = STYLE_SCALE[cfg.scale_type]

  has_minor_third = pc_weight[(best_root + 3) % 12] > 0

  if not has_minor_third and style_default ∈ {"minor","dorian"}:
      scale_key = "major"
  elif has_minor_third and style_default == "major":
      scale_key = "minor"
  else:
      scale_key = style_default
```

`_global_scale` construction:

Enumerate all notes from MIDI 48 to 84 (octaves 3–6) that conform to `(note - best_root) % 12 ∈ scale_intervals`. Sort and deduplicate.

Helper methods derived from key state:

```
engine:_nearest_scale_note(midi_note, scale_override)
      -- linear search, minimum |n - midi_note|

  engine:_scale_degree_note(degree, octave, semitone_offset)
      -- maps scale degree (1-based) to MIDI note

  engine:_build_modulated_scale(semitones)
      -- constructs _global_scale for (root + semitones) % 12
```

---

11. Core Motif Builder (`Engine:_build_core_motif`)

```
engine:_build_core_motif()
```

Generates a single hook phrase used as the A-phrase identity throughout the song.

Step 1 — Rhythm pattern selection

Picks one of 6 hardcoded rhythmic templates (up to 6 attack positions within a 16-step bar):

```
{0,2,3,6,8,12}, {0,3,4,8,11,14}, {0,2,6,8,9,12},
  {0,4,6,11,12,14}, {0,1,4,8,11,13}, {0,2,4,6,8,10}
```

Step 2 — Contour pattern selection

Uses the first two `_sig_intervals` values (style-specific) mapped to scale degrees via a semitone→degree table:

```
iv_to_deg: 1→1, 2→1, 3→2, 4→2, 5→3, 6→3, 7→4, 8→5, 9→5, 10→6, 11→6, 12→7
```

Five contour templates use `d1` and `d2` as relative degree movements.

Step 3 — Note sequence

Starting from a random base degree (3, 5, or 6) and octave (4 or 5), each contour step is accumulated and clamped, then mapped to a MIDI note via `_scale_degree_note`.

Stored in `engine._core_motif`:

```
{
      rhythm   = {step positions},
      notes    = {MIDI notes},
      base_oct = integer,
      contour  = {degree offsets},
      base_deg = integer
  }
```

Variation cycle (tracked by `_hook_variation`, advances every 3 plays):

`_hook_variation`	Modification
0	Original
1	+2 semitone shift
2	+12 semitones (octave up)
3	+12 semitones + diatonic harmony on even positions

---

12. Secondary Dominant Insertion

```
bars = engine:_apply_secondary_dominants(bars)
```

Iterates over the original bars array. For each adjacent pair `(bar[i], bar[i+1])`:

Diminished passing chord (requires `options.diminished_passing`):

```
condition: (next_root - bar.root) % 12 == 2  AND  rand() < 0.45
  action: insert dim7 chord rooted at (bar.root + 1) % 12
```

Secondary dominant (always evaluated independently):

```
root_diff = min(|bar.root - next_root|, 12 - |bar.root - next_root|)
  sec_dom_root = (next_root + 7) % 12

  condition: root_diff >= 3  AND  sec_dom_root != bar.root  AND  rand() < 0.35
  action: insert dominant 7th chord at sec_dom_root
```

Tritone substitution (primary chord is dominant of key):

```
condition: is_dominant(bar.root, next_bar.root, global_root)  AND  rand() < 0.35
  action: insert dominant 7th chord at (next_bar.root + 1) % 12
```

The three conditions are tested in separate `if` blocks (not `elseif`), allowing stacking.

---

13. Melody Generator (`Engine:_gen_melody`)

```
melody = engine:_gen_melody(
      chord, is_chorus, bar_num, section, phrase_role,
      chorus_index, is_dominant_chord, semitone_offset,
      strip_level, density_map, macro_dyn
  )
```

Returns a sparse table `melody[step] = {note, dur, flags...}` for steps 0–15.

Phrase role system

Role	Assignment	Behaviour
`"A"`	`bar_num % 4 == 0`	Stamp core motif; cache result
`"Ap"`	`bar_num % 4 == 1`	Replay cached A phrase verbatim
`"B"`	`bar_num % 4 == 2`	Contrasting bridge phrase (lower register)
`"A2"`	`bar_num % 4 == 3`	Modified A phrase + forced resolution at step 14

When `options.q_and_a_phrasing` is active:

- A / B roles target scale degree 5 (tension)
- A′ / A2 roles target scale degree 1 (resolution)

Asphyxia rule

```
if options.phrase_breath AND _melody_play_streak >= 3:
      emit only: melody[14] = {note=target_note, dur=4, is_resolution=true}
      reset _melody_play_streak = 0
      return early
```

Base density calculation

```
base_density = (0.13 + comp × 0.036 + (is_chorus ? 0.08 : 0)) × dyn
  if strip_level > 0: base_density × 0.40
```

Melodic contour shapes

Contour	Profile
`"rise"`	Linear ramp from `target_note - 7` to `target_note`
`"fall"`	Linear ramp from `target_note + 5` down
`"peak"`	Rise to `target_note + 5` at step 10, then fall
`"wave"`	`target_note + round(sin(step × 0.8) × 3)`
`"flat"`	Constant `target_note`

Per-step probability

```
step_mult:
    s <= 7   → 1.10   (front half, more dense)
    s <= 11  → 0.85
    s <= 15  → 0.40   (end of bar, sparse)
    s ∈ {3,7,11} → +15% bonus


  note_probability = clamp(
      (base_density + (is_dominant_chord ? 0.25 : 0)) × step_mult,
      0.05, 0.95
  )
```

Chromatic passing notes

On odd steps, 20% probability of using a chromatic semitone (±1) from the previous note if that note is not already in the current scale. Flagged as `is_passing = true`.

Velocity shaping on melody notes

Cumulative multipliers applied to `vel_base = 90 × dyn`:

Flag	Multiplier
`is_resolution`	×1.10
`is_passing`	×0.65
`is_motif`	×1.05
`is_climax`	×1.20
`tension_vel_mult`	from chord flag (1.15 if dominant)
`dynamics_contour`	`0.85 + ((note - 48) / 36) × 0.30`
`metric_velocity_hierarchy`	step-based (see section 21)

Grace notes

Condition: `options.smart_grace_notes AND step % 4 == 0 AND rand() < 0.40`

```
grace_note = {note - 1, vel - 30, start = ppq - 30, end = ppq - 8}
```

Motif notes on beat positions also have a 25% chance of a similar ornament with slightly different timing (−35 / −7 ticks).

---

14. Counter-Melody Generator (`Engine:_gen_counter_melody`)

```
counter = engine:_gen_counter_melody(
      melody, chord, section, semitone_offset, strip_level, density_map
  )
```

Guards:

- Only active in Chorus, Bridge, Verse.
- `strip_level > 0` → returns `{}`.
- `rand() > 0.70` → returns `{}` (30% generation probability).

Strict counterpoint mode (`options.strict_counterpoint`)

When the phrase cache for this section exists and `rand() < 0.65`:

Retrograde (50% of strict cases):

```
for each cached step s:
      target_step = 15 - s
      if density_map[target_step] != 1:
          counter[target_step] = cached_melody[s]
```

Melodic inversion (50%):

```
pivot = _scale_degree_note(5, 4, semitone_offset)
  for each cached step s:
      if density_map[s] != 1:
          diff = cached_note - pivot
          counter[s] = nearest_scale_note(pivot - diff)
```

Free counterpoint mode

Base note from scale degree 3 of the key + 4 semitones (approximately the mediant), clamped to MIDI 52–72.

Per step: `rand() < 0.55` probability. With `options.contrary_movement`, the offset direction is opposed to the main melody's local direction.

Duration of counter notes: 2 if previous step was empty, else 1.

---

15. Drum Generator (`Engine:_gen_drums`)

```
drums = engine:_gen_drums(
      is_chorus, is_pre_drop, section, local_bar, strip_level, macro_dyn
  )
```

Returns:

```
{
      kick   = [{step, swing_off, vel}],
      snare  = [{step, swing_off, vel, is_flam}],
      ghost  = [{step, swing_off, vel}],
      hat    = [{step, swing_off, note, vel, dur}],
      fills  = [{step, swing_off, vel}],
      crash  = [{step, vel}]
  }
```

Swing

```
swing_ticks = floor(cfg.swing × 240 / 100)
  swing_off   = (step % 2 == 1) and swing_ticks or 0
```

Kick

Fires at steps listed in `cfg.kick` with `rand() < drum_mult`.
Pre-drop extra kicks: steps 12, 14 (even only) at reduced velocity.

Snare

Fires at steps in `cfg.snare`.
Flam (if `options.drum_linear_fills`): beats 2/4 (steps 4, 12), 25% probability.
Flam = additional snare hit 18 ticks early at `vel - 40`.

Ghost notes

Odd steps not coinciding with snare, probability `0.18 × comp/10 × drum_mult`.
Velocity: `gauss(22 × energy, 1.3)`.

Hi-Hat / Ride logic

Even steps (downbeats): closed hat (42) or ride (51/53 if `use_ride`).
Open hat (46) replaces closed on steps listed in `cfg.hat_open_steps`.

If `options.groove_pocket_hat`:

```
base_vel × 1.1 on beats (step % 4 == 0)
  base_vel × 0.8 off beats
  +12 if kick is also active on this step
```

If `options.midi_sidechain_drums AND snare step`:

```
hat_vel × 0.55
```

Odd steps: off-beat hi-hat at lower velocity (40), with `swing_ticks` applied.

Beat drop

```
is_beat_drop = options.beat_drop AND is_pre_drop AND rand() < 0.35
  if is_beat_drop: skip all events at step >= 12
```

Euclidean percussion (`options.euclidean_percussion`)

```
function generate_euclidean(k, n)
      -- Bresenham/Bjorklund distribution
      bucket = 0
      for i = 0, n-1:
          bucket += k
          if bucket >= n: bucket -= n; sequence[i] = true
          else:           sequence[i] = false
```

`k` chosen from {3, 5, 7} with weights {0.4, 0.4, 0.2}. Random rotation 0–4. Note: MIDI 37 (side stick).

Crash

Fires at step 0 of `local_bar == 0` (first bar of each section).

Pre-drop snare roll

12-step crescendo roll from step 12.0 to 15.75 (sub-step positions), velocities from 62 to 116.

Random fills

Outside of pre-drop, 28% chance of a 3-note fill on steps {13, 14, 15} (non-snare steps only).

---

16. Bass Generator (`Engine:_gen_bass`)

```
bass = engine:_gen_bass(
      chord, next_chord_root, is_chorus, section,
      semitone_offset, bar_idx, density_map
  )
```

Returns a sparse table `bass[step] = pitch_or_struct`.

Root note derivation

```
root    = ((chord.root + semitone_offset) % 12) + 36
  fifth   = ((chord.root + semitone_offset + 7) % 12) + 36
  third   = ((chord.root + semitone_offset + (minor ? 3 : 4)) % 12) + 36
  seventh = ((chord.root + semitone_offset + 10) % 12) + 36
```

All notes anchored in octave 2 (MIDI 36 = C2).

Approach note

The note at step 15 leading into the next chord:

- `options.diatonic_bass_approach`: nearest diatonic note ±2 semitones from next root.
- `options.bass_passages`: 65% chance of using the next chord's third instead of chromatic approach.
- **Default:** chromatic semitone above or below next root.

Pattern selection

Two tiers: **sparse** (Verse, Bridge, Intro, Outro) and **full** (all others), with per-style patterns:

Style	Full pattern steps
Trap	0, 8, 10, 12
Lo-Fi	0, 3, 6, 8, 12, 14
R&B	0, 2, 3, 6, 8, 10, 11, 14, 15
Techno / House	0, 2, 4, 6, 8, 10, 12, 14, 15
Rock	0, 4, 6, 8, 12, 14, 15
Pop (default)	0, 2, 4, 6, 8, 10, 12, 13, 14, 15

Chorus octave shift

```
if is_chorus: all bass[s] += 12
```

Density-map fill (odd bars)

If `bar_idx % 2 == 1` and step is free in `density_map`, 25% chance of adding root/fifth/third filler notes.

Ghost notes (`options.bass_ghost_notes`)

For each bass note at step `s > 0` where `bass[s-1]` is empty:

```
35% chance: bass[s-1] = {note=bass[s], is_ghost=true}
```

Ghost: velocity ~30, duration = 15% of PPQ.

Octave jumps (`options.bass_octave_jumps`)

Steps {6, 14, 15}: 35% chance each to write `{note = note + 12, is_pop = true}`.
Pop note: velocity ~105, duration = 25% of PPQ.

Harmonic pedal (`options.harmonic_pedal`, Pre-Chorus only)

Locks bass to global key root for all even steps 0–14. Approach note only on step 15 of odd bars.

---

17. Pad Generator (`Engine:_gen_pad`)

```
pad = engine:_gen_pad(chord, section, semitone_offset, strip_level, macro_dyn)
```

Returns array of `{step, notes[], vel, dur, anticipated}` records.

Step grid by section

Condition	Steps
`strip_level >= 1` OR Intro/Outro	`{0}`
Verse / Bridge	`{0, 8}`
`options.polyrhythmic_pad`	`{0, 3, 6, 9, 12, 15}` (3-vs-4)
Default	`{0, 4, 8, 12}`

Pitch-class set construction

Start from `chord.pcs`, apply `semitone_offset`:

```
pcs[i] = (pc + semitone_offset) % 12
```

Color extensions added by section:

- Verse / Bridge: add major 9th `(root + 14) % 12` (if not already present, excludes dim/aug/power chords).
- PreChorus + minor chord: add 11th `(root + 17) % 12`.
- Chorus + major/dom chord: add major 7th or 13th `(root + 21) % 12`.

Voice leading

```
voicing = best_voicing(pcs, engine._prev_voicing, options.optimal_voice_leading)
```

`best_voicing` evaluates all chord inversions at octaves 4 and 5, returning the arrangement with minimum total distance from `_prev_voicing`. If `optimal_voice_leading` is false, defaults to root-position voicing at octave 5.

Drop voicings (`options.voicing_opened`, Chorus only)

```
voicing = make_drop_voicing(voicing, (#voicing >= 4) and 3 or 2)
```

Drop 2: second-highest voice lowered one octave.
Drop 3: third-highest voice lowered one octave.

Harmonic anticipation (`options.harmonic_anticipation`)

Step 0 of each pad block: 40% probability of moving the event to `b_off - 240` (one beat early), with `dur_ppq += 240` to compensate.

---

18. Voice Leading Engine

`get_chord_inversions(pcs)`

Generates root position + 1st inversion + 2nd inversion for triads and larger chords:

```
root:  [pc0, pc1, pc2]
  1st:   [pc1, pc2, pc0+12]
  2nd:   [pc2, pc0+12, pc1+12]
```

All sorted ascending.

`best_voicing(chord_pcs, prev_notes, use_optimal_lead)`

```
if not use_optimal_lead or prev_notes empty:
      return [pc + 60 for pc in chord_pcs]   -- root position, octave 5

  for each inversion × octave {4, 5}:
      candidate = inversion notes + oct*12 (sorted)
      dist = Σ |candidate[i] - prev_notes[i]|
           + |#candidate - #prev_notes| × 7   -- size penalty
      track minimum dist → best_v
  return best_v
```

---

19. Song Assembly (`Engine:generate_song`)

```
song_data, key_name = engine:generate_song(chords_text)
```

Fixed structure template

```
structure = {
      {"Intro",      repeats=2, modulated=false, strip=2},
      {"Verse",      repeats=2, modulated=false, strip=0},
      {"PreChorus",  repeats=1, modulated=false, strip=0},
      {"Chorus",     repeats=2, modulated=false, strip=0},
      {"Verse",      repeats=2, modulated=false, strip=0},
      {"PreChorus",  repeats=1, modulated=false, strip=0},
      {"Chorus",     repeats=2, modulated=false, strip=0},
      {"Bridge",     repeats=1, modulated=true,  strip=1},
      {"Chorus",     repeats=2, modulated=false, strip=0},
      {"Outro",      repeats=1, modulated=false, strip=2},
  }
```

Bridge: `is_modulated = true` → `semitone_off = engine._modulation_st` (2 semitones).

Macro dynamics

If `options.macro_dynamics`:

```
progress = local_bar / (total_bars_in_section - 1)


  Outro:  macro_dyn × (1.1 - 0.6 × progress)   -- fade out
  Intro:  macro_dyn × (0.6 + 0.5 × progress)   -- fade in
  Others: macro_dyn × (0.85 + 0.3 × progress)  -- gentle swell
```

Density map

A per-bar `density_map[0..15]` table is shared across all generators. The melody generator writes `1` at each occupied step. The counter-melody and bass generators use it to avoid collisions.

Bridge borrowing

```
if section_type == "Bridge":
      c = borrow_chord(chord, rng)
```

`borrow_chord` picks from parallel-mode harmonies: flat VII, flat VI, flat III relative to the current chord root. Intervals default to minor quality if not specified.

Bar record written to `song_data`

```
{
      section         string
      chorus_index    integer or nil
      is_pre_drop     boolean
      dyn             float
      strip_level     integer
      melody          sparse table [0..15]
      counter_melody  sparse table [0..15]
      drums           {kick,snare,ghost,hat,fills,crash}
      bass            sparse table [0..15]
      pad             array of pad records
  }
```

---

20. MIDI Writer (`write_midi_multi_track`)

```
write_midi_multi_track(song_data, engine, clean_old_tracks)
```

Track creation

Five tracks are inserted starting at `reaper.CountTracks(0)`:

```
TRACKS_DEF = {
      {name="[MIDISTRUCT] DRUMS",       r=200,g=50, b=50 },
      {name="[MIDISTRUCT] BASS",        r=50, g=140,b=220},
      {name="[MIDISTRUCT] PAD-CHORDS",  r=140,g=60, b=220},
      {name="[MIDISTRUCT] MELODY",      r=50, g=200,b=100},
      {name="[MIDISTRUCT] COUNTER-MEL",r=220,g=140,b=40 },
  }
```

Track colour is encoded as `r + g×256 + b×65536 + 0x1000000`.

Section → MIDI item mapping

`sections_list` is built by scanning `song_data` for section boundary transitions. One MIDI item per `(section, chorus_index)` pair per track is created via `reaper.CreateNewMIDIItemInProj`.

PPQ offset of each bar within its section:

```
bar_ppq_offset = (bar_index - section.start_bar) × 16 × PPQ
```

MIDI sort

After all notes and CCs are written, `reaper.MIDI_Sort(take)` is called on every take. `reaper.MIDI_DisableSort(take)` is called before writing to batch operations.

Drum channel

All drum `MIDI_InsertNote` calls use channel **9** (0-indexed), i.e., `0x99` MIDI status byte.

---

21. Timing & Humanization Pipeline

For each MIDI event, timing is computed as:

```
base_ppq = b_off
           + floor(step × PPQ)
           + floor(rubato(step, rubato_intensity) × PPQ / 120)
           + push_pull_ticks                          -- SECTION_PUSH_PULL[section]
           + organic_shift                            -- GROOVE_MAPS offset
           + swing_off                                -- drums only

  ppq = Humanizer:timing(base_ppq, tightness, step, deterministic)
  ppq = max(b_off, ppq)                               -- clamp to bar start
```

`push_pull_ticks` (applied to all instruments including drums):

Section	Ticks
Chorus	−12
PreChorus	−6
Verse	+10
Bridge	+8
Outro	+5
Intro	0

Note duration calculation (melody)

```
dur_ppq = floor(note.dur × PPQ × engine.legato)
  if is_passing: dur_ppq = floor(PPQ × 0.35)
  note_end = max(ppq + 20, ppq + dur_ppq)
```

Metric velocity hierarchy

Applied to melody and bass when `options.metric_velocity_hierarchy`:

Step	Multiplier
0 (beat 1)	×1.15
8 (beat 3)	×1.02
4, 12 (beats 2, 4)	×0.94
Odd steps	×0.72
Other even	×0.85

---

22. CC Automation Pipeline

Pad CC11 (Expression / Swell)

Written per pad step, over the duration `dur_ppq` of the chord:

```
attack_steps  = floor(steps_count × 0.65)
  release_steps = steps_count - attack_steps

  attack  phase: val = swell_start + (swell_peak - swell_start) × (1 - exp(-3t))
  release phase: val = swell_peak  - (swell_peak - swell_end)   × t

  step_ppq = 60 ticks (25ms at 240 PPQ / ~120 BPM equivalent)
  swell_start = 40
  swell_peak  = clamp(floor(100 × dyn), 40, 127)
  swell_end   = floor(swell_peak × 0.82)
```

Pad CC74 (Timbre / Filter) — `options.cc_automation_curves`

Written once at bar start:

```
filter_base = clamp(floor(40 + dyn × 60), 10, 127)
  MIDI_InsertCC(t_pad, b_off, 0xB0, ch=0, CC=74, val=filter_base)
```

In pre-drop bars: per-step ramp 0→127 written alongside.

Pad CC1 (Modulation) — pre-drop tension

```
for step = 0 to 15:
      tension_val = floor((step / 15) × 127)
      MIDI_InsertCC(t_pad, b_off + step×PPQ, 0xB0, CC=1, val=tension_val)
```

Melody CC1 (Vibrato)

On sustained notes (`dur_ppq >= 2 × PPQ`, not passing):

```
vib_start = ppq + floor(dur_ppq / 3)
  vib_depth = is_chorus ? 18 : 10

  CC1 = 0   at vib_start
  CC1 = vib_depth at vib_start + floor(dur_ppq / 3)
  CC1 = 0   at note_end - 20
```

Melody Pitch Bend Sag — `options.pitch_bend_sag`

Condition: `dur_ppq >= 3 × PPQ AND step >= 8`

```
bend_start = note_end - floor(dur_ppq × 0.4)

  Reset to center:     PB = 8192 at bend_start
  Quadratic fall over 6 steps:
      val = 8192 - floor(1200 × (i/6)²)   (i = 1..6)
  Reset to center:     PB = 8192 at note_end + 1
```

Pitch bend encoded as 14-bit value split into LSB/MSB:

```
function insert_pitch_bend(take, ppq_pos, val)
      lsb = val % 128
      msb = floor(val / 128)
      MIDI_InsertCC(take, ppq_pos, 0xE0, ch=0, lsb, msb)
```

---

23. Section Region Insertion

```
insert_section_regions(song_data, start_pos, bpm)
```

One coloured REAPER region per contiguous section run:

```
spb = 4 × (60.0 / bpm)   -- seconds per bar

  on section change:
      AddProjectMarker2(
          proj=0, isrgn=true,
          rgn_start = start_pos + (first_bar - 1) × spb,
          rgn_end   = start_pos + (current_bar - 1) × spb,
          name = section_name [+ " N" if chorus],
          color = reaper_color(SECTION_REGION_COLOR[section])
      )
```

---

24. Settings Persistence

All settings saved/loaded via `reaper.SetExtState / GetExtState("MIDISTRUCT", key, value, persist=true)`.

Saved keys

Key	Type	Default
`s1`	int	1
`s2`	int	0
`mix`	int	0
`comp`	int	5
`seed`	int	0
`chords`	string	`"Am:4 F:4 C:4 G:4"`
`clean_tracks`	"0"/"1"	"1"
`preset_idx`	int	2
`opt_<name>`	"0"/"1"	see below

Default option states

Options default to **on** (loaded with `~= "0"`) unless explicitly stored as "0":

```
bass_ghost_notes, macro_dynamics, polyrhythmic_pad, bass_passages,
  harmonic_pedal, contrary_movement, motivic_shift, smart_grace_notes,
  pitch_bend_sag, beat_drop, drum_linear_fills, dynamics_contour,
  harmonic_anticipation, phrase_breath, voicing_opened, groove_pocket_hat,
  optimal_voice_leading, laidback_snare, metric_velocity_hierarchy,
  modal_interchange
```

Options default to **off** (loaded with `== "1"`):

```
deterministic_groove, diatonic_bass_approach, q_and_a_phrasing,
  midi_sidechain_drums, humanized_strumming, bass_octave_jumps,
  euclidean_percussion, diminished_passing, organic_groove_maps,
  strict_counterpoint, cc_automation_curves
```

---

25. ReaImGui Render Loop

```
reaper.defer(render_imgui)
```

`render_imgui` is the top-level deferred function. It calls:

```
reaper.ImGui_Begin(ctx, title, true)   -- open=true
```

Returns `visible, open`. If `open == false` (window closed by user): `save_settings(...)` and do **not** re-defer → script terminates.

If `visible == true`: render all 7 collapsible sections and the generate button.

Generate button:

```
reaper.Undo_BeginBlock()
  reaper.PreventUIRefresh(1)
      write_midi_multi_track(song, engine, clean_tracks)
      log_history(...)
  reaper.PreventUIRefresh(-1)
  reaper.UpdateArrange()
  reaper.Undo_EndBlock("MIDISTRUCT V11.0 | ...", -1)
```

The entire generation is wrapped in a single REAPER undo block.

---

26. Data Structures Reference

Bar record (input to generators)

```
{
      root     int     -- pitch class 0–11
      pcs      table   -- array of pitch classes
      quality  string  -- "maj","min","dom","dim","aug","sus","pow"
      name     string  -- original string token
      is_secondary_dominant boolean (optional)
  }
```

Melody step record

```
{
      note             int      -- MIDI note 48–84
      dur              int      -- duration in sixteenth units (1–4)
      harmony          int?     -- optional second note
      is_passing       boolean
      is_motif         boolean
      is_climax        boolean
      is_resolution    boolean
      is_cached        boolean
      has_grace_note   boolean
      tension_vel_mult float?
  }
```

Drum hit record

```
{
      step      int       -- 0-15
      swing_off int       -- additional swing ticks
      vel       int       -- MIDI velocity 1-127
      note      int?      -- override default MIDI note
      dur       int?      -- override default duration ticks
      is_flam   boolean?  -- snare only
  }
```

Pad step record

```
{
      step        int      -- 0-15 (or -1 if anticipated)
      notes       table    -- array of MIDI notes
      vel         int
      dur         float    -- in quarter-note units
      anticipated boolean
  }
```

Bass step value

Either an integer (raw MIDI note) or a table:

```
{
      note     int
      is_ghost boolean   -- ghost note: vel ~30, dur 15% PPQ
      is_pop   boolean   -- octave jump: vel ~105, dur 25% PPQ
  }
```

---

27. Constants Reference

Constant	Value	Notes
`PPQ`	240	Pulses per quarter note
`modulation_st`	2	Bridge semitone shift
`grace_note_offset`	−30 / −8 ticks	Start/end relative to main note
`motif_grace_offset`	−35 / −7 ticks	Motif variant
`climax_step`	10	Default climax position in bar
`swell_step_ppq`	60 ticks	CC11 resolution
`snare_laidback`	+16 ticks	Laidback snare offset
`density_switch`	0.5	Blend ratio threshold for table fields
`vib_center`	8192	Pitch-bend center (14-bit)
`vib_sag_depth`	1200	Max pitch-bend sag in cents (14-bit units)
`asphyxia_threshold`	3	Consecutive dense bars before breath
`sec_dom_prob`	0.35	Secondary dominant insertion probability
`dim_pass_prob`	0.45	Diminished passing chord probability
`tritone_sub_prob`	0.35	Tritone substitution probability
`modal_int_prob`	0.35	Modal interchange probability
`counterpoint_prob`	0.70	Counter-melody generation gate
`strict_cp_prob`	0.65	Strict counterpoint use probability

28. Known Constraints & Design Decisions

Fixed song structure. The 10-section template is hardcoded in `generate_song`. There is no API to modify it at runtime without editing the source.

Single PPQ resolution (240). All timing arithmetic assumes 240 PPQ. Changing this constant requires updating every fixed-offset calculation (`vib_center`, `grace_note_offset`, `swell_step_ppq`, etc.).

Swing applies to drums only. The `swing_off` field is computed per drum hit. Melody, bass, and pad timing go through `Humanizer:timing` with their own sigma — they do not receive swing. This is intentional for genre authenticity.

Chord quality detection is longest-match, not regex-based. The parser uses a chain of `string.find` calls in priority order. Compound qualities like `"m9"` must be checked before `"9"` and `"m"` to avoid false matches.

`_global_scale` **is rebuilt on every** `_infer_key` **call.** There is no cache invalidation — calling `generate_song` multiple times on the same Engine instance would re-infer and rebuild. In practice, `Engine.new` is called fresh for each generation.

MIDI*DisableSort* / *MIDI*Sort pattern. All takes have sort disabled before note insertion and sorted once at the end. This avoids O(n²) sort costs during bulk insertion.

Bridge chord borrowing always replaces the original chord. The borrowed chord is drawn from `borrow_chord` which picks from three parallel-mode candidates. The original chord's root and quality are discarded, producing unconventional Bridge harmonies by design.

Density map is pre-filled to 0, not nil. Steps are initialized `for s=0,15 do density_map[s]=0 end` so counter-melody and bass can test `density_map[s] == 1` (occupied) vs `== 0` (free) vs `nil` (not initialized). These are equivalent for the bass filler logic but not for the counter-melody checks.

`bar_ppq_offset` **linear scan.** For each bar, the function iterates `sections_list` to find the matching section. For typical song lengths (≤ 50 bars) this is negligible, but it scales O(bars × sections).
